



Assert



Assert in Java

- Declarações de código que permitem que você verifique se uma condição é atendida (ou não)
- Ajuda a controlar erros de código durante o desenvolvimento
- Eles são equivalentes aos ifs "típicos" que colocamos em todo o código para verificar os valores ou se ele é alcançado ou não (desde que não vamos usar o debugger)
- Eles podem ser ativados / desativados durante a execução do aplicativo
 - Eles são ativados ao executar a classe principal (dando a ele a opção -ea ou -enableassertion)



Se executarmos o seguinte código:

```
...
String texto= "Isto é um texto";
...
...
assert texto== null : "A String é nula";
...
// Também se pode especificar uma condição sem mensagem
// assert texto == null;
...
```

Obteríamos a seguinte saída de tela (se as asserções estiverem habilitadas):

```
run: Exception in thread "main" java.lang.AssertionError: A String é nula at
com.tokioschool.prueba.JavaApplication5.main(JavaApplication5.java:29)
/Users/Santi/Library/Caches/NetBeans/8.2/executor-snippets/run.xml:53: Java returned: 1 BUILD FAILED (total time: 0 seconds)
```



Coleções e genéricos



O que são genéricos?

- São classes definidas onde um tipo de dado é parametrizado
- Permite avançar para erros de tempo de compilação (desenvolvimento) de tipos de dados que ocorreram anteriormente em tempo de execução
 - `List integerList1 = new ArrayList()` vs `Lis<Int> integerList2 = new ArrayList()`
- O objetivo é ajudar o programador a controlar falhas como esta:
 - `String string = integerList1.get (0)` // falha de compilação
 - `String string = (String) integerList1.get (0)` // Não falharia na compilação, mas falharia na execução
 - `String string = integerList2.get (0)` // falha de compilação
 - `String string = (String) integerList2.get (0)` // falha de compilação
 - `int num = integerList2.get (0);` // Direito. Existe apenas uma opção válida e que os tipos correspondem
- É algo que só existe em tempo de compilação. Ao gerar o código (.class), o Java adiciona todos os casts necessários para que o código pode ser executado sem problemas



Classes genéricas

- Já existem classes genéricas em Java
- As classes usadas para armazenar coleções são todas genéricas: `List <>`, `ArrayList <>`, `HashMap <,>`, ...
- Eles são sempre definidos indicando o tipo de dados para os quais desejam ser implementados
- Este tipo de dado terá sido deixado como genérico na definição da classe (então vemos um exemplo):
 - `List <String> stringList = new ArrayList <> ()`
 - `Map <String, Product> producList = new HashMap <> ()`
- Existe uma convenção para definir o tipo genérico de uma classe genérica:
 - E para definir um elemento de uma coleção
 - K para definir uma chave em um mapa
 - V para definir um valor em um mapa
 - T, U para definir os tipos de dados (classes)



O curinga ?

- Você também pode usar o curinga ? para indicar que qualquer tipo é aceito como parâmetro:

```
public void doSomething (ArrayList <?> aColecion) {  
    ...  
}
```

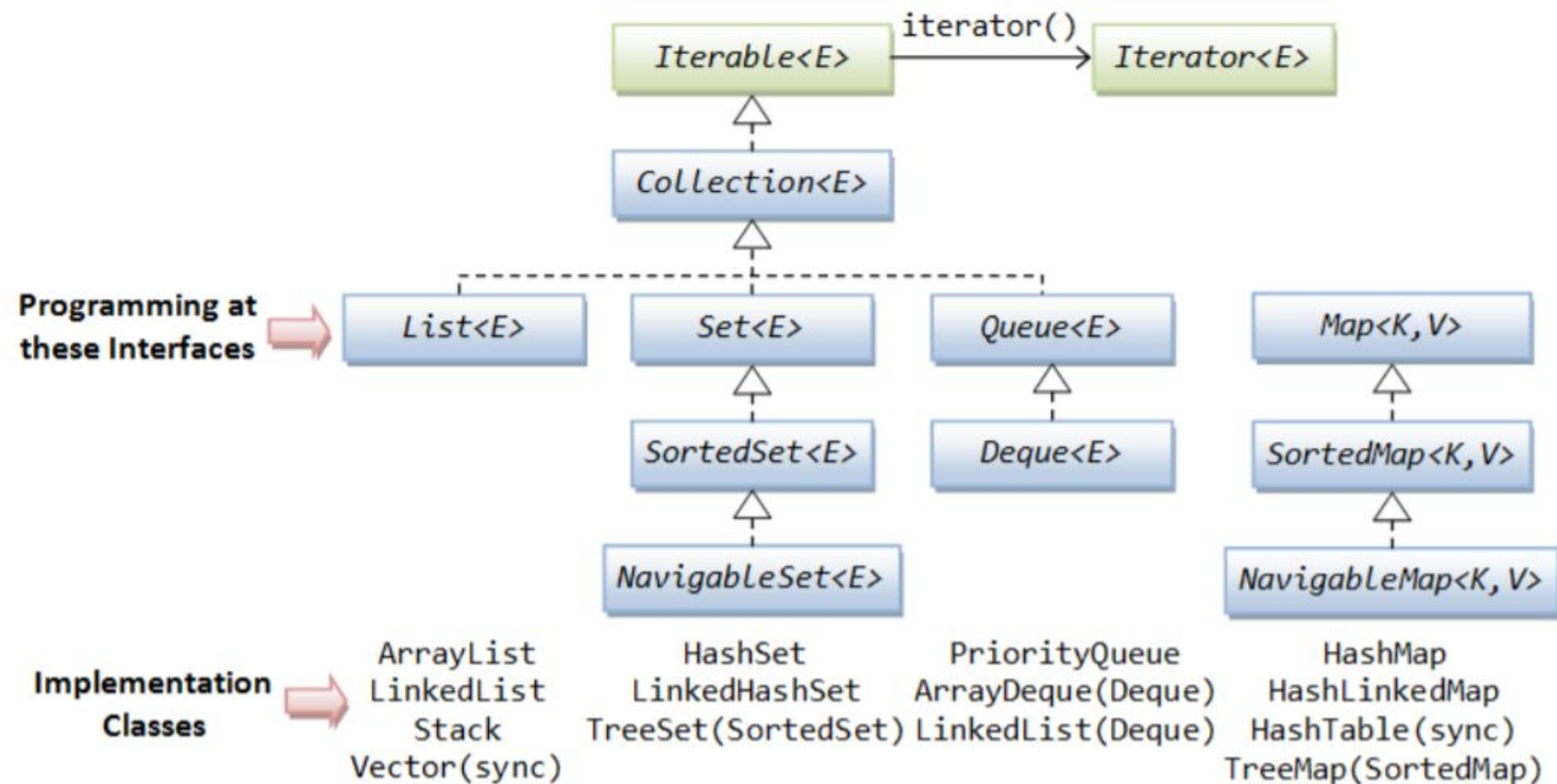
- E também para indicar que aceitamos tipos que se estendem ou estão acima de uma determinada classe:

```
public void addAnimals (ArrayList <? extends Animal> animais) {  
  
    public void addAnimals (ArrayList <? super Animal> animais) {
```

- Deve-se levar em consideração que, sem o uso de curingas, Java não aceitará como tipos genéricos aqueles que, por exemplo, herdam daquela classe genérica. Para esses casos é necessário usar o curinga ?



A interface Collection em Java





Diferenças entre as coleções

JCF Class	Positioning	Ordering	Duplicates	Sample Use
TreeSet	None	Ascending	Not Allowed	Keep unique list of words from a document sorted
HashSet	None	None	Not Allowed	Keeps a unique list of words from a document to be referenced frequently
ArrayList	Numerical	None	Allowed	Keeps an “in-memory” lookup table of static data
LinkedList	Numerical	Insertion Order	Allowed	Keeps a list of words that change frequently
HashMap	Object as key	None	Unique Keys	Maps words to word frequency
TreeMap	Object as key	Ascending	Unique Keys	Maps sorted words to word frequency



Aspectos a ter em conta quando trabalhar com coleções

- Estude bem que tipo de coleção é mais interessante para cada caso (nem sempre use ArrayList apenas para tudo)
- Implementar / estender os métodos da classe Object nas classes que você projeta:
 - hashCode()
 - equals()
 - toString()
- Aprenda a usar algumas interfaces interessantes para usar com coleções:
 - Comparador
 - Comparável



Dúvidas?

Tokio.



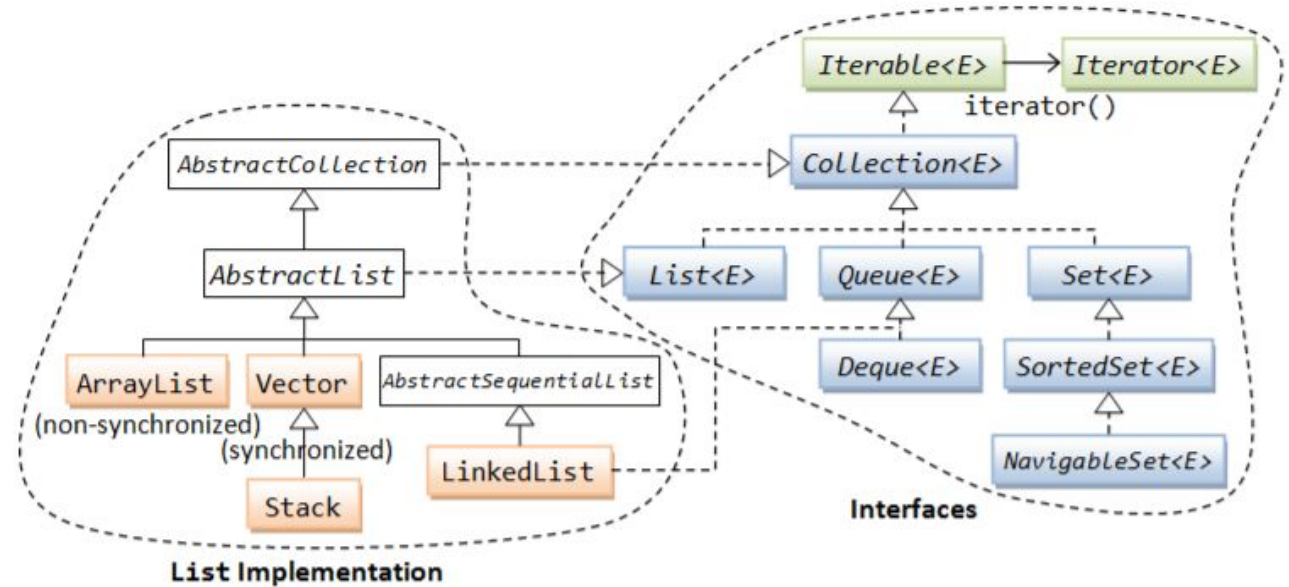
Collection

- É a interface pai de todas as coleções Java
- Define uma série de métodos que qualquer coleção deve implementar
 - `boolean add(E e)`
 - `boolean addAll(Collection c)`
 - `void clear()`
 - `boolean contains(E e)`
 - `boolean isEmpty()`
 - `Iterator<T> iterator()`
 - `boolean remove(E e)`
 - `boolean removeAll(Collection c)`
 - `int size()`
 - `Object[] toArray():`
 - `<T> T[] toArray(T[] a)`



List

- É a interface pai de todas as coleções do tipo Lista
- Define uma série de métodos que qualquer coleção deve implementar
 - E get (i)
 - int indexOf (e)
 - void sort(Comparator<? super E> c)





ArrayList

- Implementar a interface de lista
- É o tipo de coleção mais genérico
- Define uma coleção de elementos sem quaisquer características especiais
- Os elementos podem ser obtidos a partir da posição onde foram adicionados

LinkedList

- Implementar a interface de lista
- Usado para definir estruturas FIFO (First In First Out), também conhecidas como filas
- Define métodos que permitem, diretamente, acessar / excluir o primeiro elemento de uma lista (o primeiro elemento da fila)

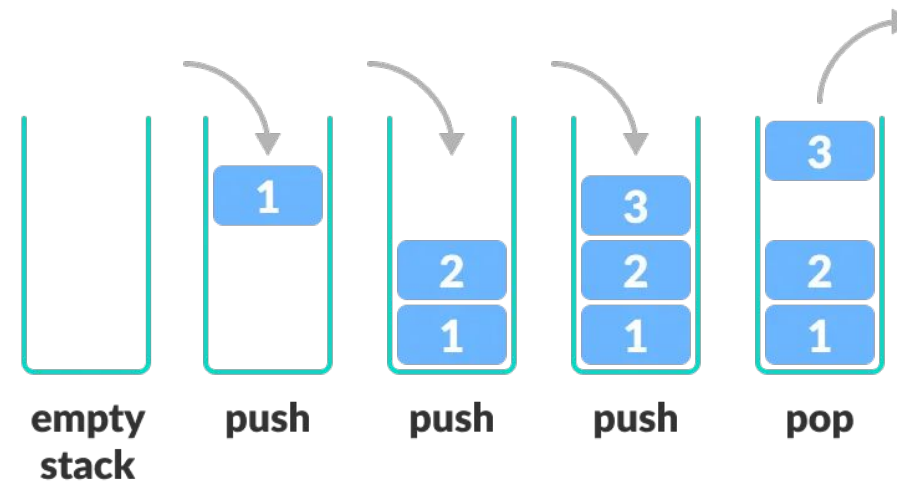
Vector

- Implementar a interface de lista
- É a mesma implementação que ArrayList, mas sincronizada para uso em ambientes simultâneos. Por exemplo, O vetor dobra o tamanho de sua matriz quando seu tamanho é aumentado. Mas, ArrayList aumenta pela metade de seu tamanho.



Stack

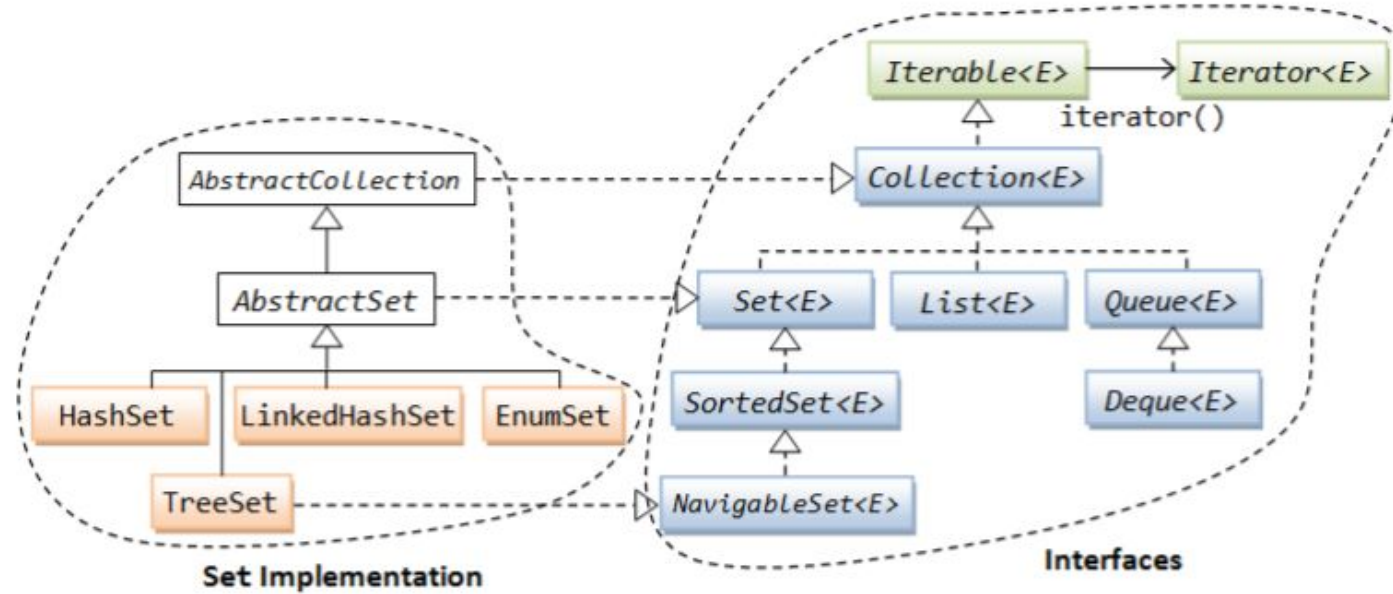
- Implementar a interface de lista
- Permite que você trate uma lista como se fosse uma estrutura LIFO (Last In First Out), também conhecida como pilhas.
- Definir métodos (push e pop) para pegar e soltar diretamente do final da lista (que seria o topo da pilha)





Set

- Por definição, não aceita elementos duplicados
- Não define novos métodos, apenas implementa Collection e evita que elementos duplicados existam





HashSet

- Desempenho consistente para as operações mais comuns
- Não garante a ordem dos elementos da coleção

LinkedHashSet

- Mantém uma lista duplamente vinculada para que um iterador possa ser obtido com a ordem em que os elementos foram adicionados à coleção
- O desempenho é ligeiramente inferior ao de HashSet

EnumSet

- É um conjunto especializado para enumerações

TreeSet

- Garante a ordem dos elementos
- Seus elementos são sempre mantidos em uma ordem natural ou por um Comparador fornecido em seu construtor
- Seu desempenho é o mais baixo de todos os conjuntos